Practical 7: Import and Transform RetailPulse Data in Power BI

Related topic: Power BI Interface, Data Import and Transformation (Unit 3)

Name:		Roll No.:		Date:	

Note on figures and data: the tables and visuals in this report were generated from a sample RetailPulse dataset (150 sales orders, 8 products, 8 stores, 4 regions, Jan 2024 - Dec 2025) built to match the lab manual's schema. They reproduce what Power BI Desktop shows for the steps. If your workbook differs, your numbers will differ; paste your own Power BI screenshots into the marked boxes before submitting.

Objective

Explore the Power BI Desktop interface, import the RetailPulse data from a workbook, and clean and shape it using Power Query.

Materials Required

Power BI Desktop (free version); RetailPulse_Sales.xlsx containing Sales, Product and Store tables.

Procedure and Observations

Step 1 - Explore the interface.

• **Report view** (top icon on the left bar): canvas for building visuals.

• **Table view**: shows the rows of the loaded tables.

• **Model view**: shows tables and relationships.

• **Ribbon** (Home, Insert, Modeling, View, Help) holds the commands; the **Data pane** lists tables and fields, the **Visualizations pane** holds visual types and field wells, and the **Filters pane** holds visual/page/report-level filters.

Step 2 - Get data. Home > Get data > Excel workbook > select RetailPulse_Sales.xlsx. In the Navigator tick **Sales, Product and Store**, then click **Transform Data** to open Power Query Editor.

Step 3 - Set data types. In the Sales query select the column, then Transform > Data Type (or the icon in the column header):

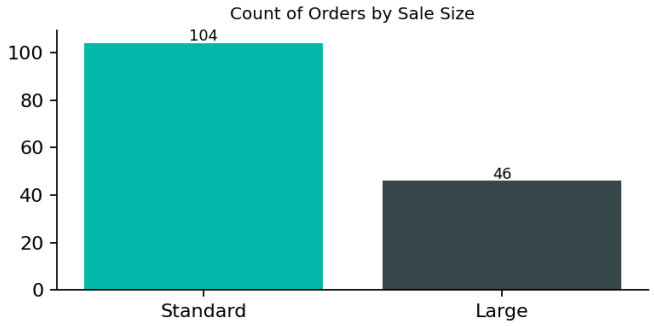
Column	Data type	Header icon
Sales Date	Date	calendar
Quantity	Whole Number	1 2 3
Sales Amount	Fixed decimal number	$
OrderID, ProductID, StoreID	Text / Whole Number (keys)	ABC / 123

Step 4 - Clean the Sales table. (a) Home > Remove Rows > Remove Blank Rows. (b) Select all columns, right-click > Remove Duplicates. (c) Select Quantity > Transform > Replace Values: Value to find *null*, Replace with *0*. (d) Double-click unclear headers and rename them: *Qty* to *Quantity* and *Sales Amt* to *Sales Amount*.

Data-quality summary for the sample sales sheet:

Check	Before	After
Total rows in Sales sheet	157	150
Completely blank rows	3	0
Duplicate rows	4	0
Null values in Quantity	5	0

Step 5 - Conditional column. Add Column > Conditional Column. New column name: **Sale Size**. If *Sales Amount* is greater than or equal to *1000* then *Large*, Else *Standard*.



Step 6 - Review and apply. Check the Applied Steps list in the Query Settings pane, then Home > **Close & Apply**. Save with File > Save as **RetailPulse_Analysis.pbix**.

Applied Steps (Sales query)

#	Applied step	What it does
1	Source	Connects to RetailPulse_Sales.xlsx
2	Navigation	Selects the Sales table
3	Renamed Columns	Qty to Quantity, Sales Amt to Sales Amount
4	Changed Type	Date, Whole Number, Fixed decimal
5	Removed Blank Rows	Deletes rows where every field is empty
6	Removed Duplicates	Keeps one copy of repeated rows
7	Replaced Value	null to 0 in Quantity
8	Added Conditional Column	Creates Sale Size (Large / Standard)

Equivalent Power Query (M) code, visible in Home > Advanced Editor:

```
let
  Source = Excel.Workbook(File.Contents("C:\Data\RetailPulse_Sales.xlsx"), null, true),
  Sales_Table = Source{[Item="Sales",Kind="Table"]}[Data],
  #"Renamed Columns" = Table.RenameColumns(Sales_Table,
      {{"Qty","Quantity"},{"Sales Amt","Sales Amount"}}),
  #"Changed Type" = Table.TransformColumnTypes(#"Renamed Columns",
      {{"Sales Date", type date},{"Quantity", Int64.Type},
       {"Sales Amount", Currency.Type}}),
  #"Removed Blank Rows" = Table.SelectRows(#"Changed Type", each not
      List.IsEmpty(List.RemoveMatchingItems(Record.FieldValues(_), {"", null}))),
  #"Removed Duplicates" = Table.Distinct(#"Removed Blank Rows"),
  #"Replaced Value" = Table.ReplaceValue(#"Removed Duplicates", null, 0,
      Replacer.ReplaceValue, {"Quantity"}),
  #"Added Conditional Column" = Table.AddColumn(#"Replaced Value", "Sale Size",
      each if [Sales Amount] >= 1000 then "Large" else "Standard", type text)
in
  #"Added Conditional Column"
```

Output: first rows of the cleaned Sales table

OrderID	Sales Date	ProductID	StoreID	Quantity	Sales Amount	Sale Size
ORD1132	2024-01-09	1	6	3	2,675.33	Large
ORD1080	2024-01-14	6	7	5	276.55	Standard
ORD1076	2024-01-19	8	8	3	121.32	Standard
ORD1141	2024-01-27	3	6	5	2,027.44	Large
ORD1055	2024-01-28	8	2	2	80.88	Standard
ORD1042	2024-01-31	7	4	4	319.62	Standard

OrderID	Sales Date	ProductID	StoreID	Quantity	Sales Amount	Sale Size
ORD1094	2024-02-01	7	7	3	224.71	Standard
ORD1059	2024-02-02	2	2	0	2,072.74	Large

Paste your Power BI screenshot here: Applied Steps pane and cleaned Sales table

Result / Expected Outcome Check

• All three tables (Sales, Product, Store) load without errors; the Data pane shows the correct type icon for every column.

• Dates are recognised as dates and amounts as numbers; 3 blank rows and 4 duplicate rows were removed and 5 null quantities set to 0.

• Sale Size labels each row: 46 Large and 104 Standard orders in the sample.

• Applied Steps records every change and re-runs automatically when the data refreshes. RetailPulse_Analysis.pbix reopens with the loaded data.

Viva-Voce Questions and Answers

a) What is the difference between Import mode and DirectQuery mode in Power BI?

In **Import** mode the data is copied into Power BI's in-memory engine; visuals are fast, all DAX/Power Query features work, but the data is only as fresh as the last refresh and the file has size limits. In **DirectQuery** mode no data is stored; each visual sends a query to the source, so data is always current and size is not limited by the model, but visuals are slower and some Power Query/DAX features are restricted.

b) What are Applied Steps in Power Query, and why are they useful?

Applied Steps is the ordered list of every transformation done on a query. Each step is recorded as M code, so it can be reviewed, reordered, edited or deleted, and the whole sequence is replayed automatically whenever the data refreshes. It gives a repeatable, auditable cleaning process without changing the source data.

c) Which of the three views would you use to check relationships between tables?

The **Model view**, which shows tables as boxes with relationship lines, cardinality (1 and *) and filter direction.

d) Why must data types be set correctly before building visuals or calculations?

Data types decide what operations are allowed and how values behave. Only numeric columns can be summed or averaged, only date columns support time intelligence and date hierarchies, and sorting, filtering and relationships depend on matching types. Wrong types (e.g. amounts as text) give errors, wrong totals, or alphabetical instead of numeric sorting.